

Chapter 28 — Union Find / Disjoint Set Union

0. What You Will Learn in This Chapter

Union Find, also called Disjoint Set Union or DSU, is a data structure for tracking connected components dynamically.

It supports two core operations:

find union

It is especially useful for undirected connectivity problems.

By the end of this chapter, you should understand:

- What Union Find is
- What disjoint sets are
- How find works
- How union works
- Why path compression matters
- Why union by rank/size matters
- How to count connected components
- How to detect cycles in undirected graphs
- How to merge accounts
- How to solve redundant connection
- How to solve provinces
- How to solve equations satisfiability
- How Union Find differs from DFS/BFS
- When to use Union Find
- When not to use Union Find

Core problems covered in this chapter:

1. Number of Connected Components in an Undirected Graph
 2. Number of Provinces
 3. Find if Path Exists in Graph
 4. Redundant Connection
 5. Accounts Merge
 6. Satisfiability of Equality Equations
 7. Graph Valid Tree
 8. Most Stones Removed with Same Row or Column
 9. Number of Operations to Make Network Connected
 10. Min Cost to Connect All Points, Kruskal preview
 11. Regions Cut By Slashes
 12. Swim in Rising Water, Union Find preview
 13. Similar String Groups
 14. Smallest String With Swaps
 15. Couples Holding Hands
 16. Number of Islands II
 17. Last Day Where You Can Still Cross
 18. Remove Max Number of Edges to Keep Graph Fully Traversable
-

1. What Is Union Find?

Union Find is a data structure that tracks groups of connected elements.
Each group is called a set or component.

It supports:

1.1 Find

Find the representative/root of the set containing an element.

```
find(x)
```

1.2 Union

Merge the sets containing two elements.

```
unite(a, b)
```

1.3 Connected

Check whether two elements are in the same set.

```
find(a) == find(b)
```

2. Why Union Find Exists

Suppose you have nodes:

```
0, 1, 2, 3, 4
```

Initially each node is alone:

```
{0}, {1}, {2}, {3}, {4}
```

After edge:

```
0 - 1
```

Groups:

```
{0,1}, {2}, {3}, {4}
```

After edge:

```
1 - 2
```

Groups:

```
{0,1,2}, {3}, {4}
```

Now:

```
connected(0, 2) == true  
connected(0, 4) == false
```

Union Find maintains this efficiently as edges are added.

3. Core Need / Why Use Union Find?

DFS/BFS can find connected components after building graph.

Union Find is useful when connectivity changes dynamically as edges are processed.

Examples:

- edges added one by one
- detect first edge that creates cycle
- merge groups
- query connectivity
- Kruskal MST
- dynamic island counting
- grouping equivalent strings/accounts

Union Find is not mainly for traversal.

It is for component management.

4. Disjoint Sets

A disjoint set collection means each element belongs to exactly one group.

Example:

$\{0,1,2\}, \{3,4\}, \{5\}$

No overlap.

Element 0 belongs only to first set.

Element 3 belongs only to second set.

Union Find stores these groups using parent pointers.

5. Parent Array

Each node points to a parent.

Initially:

`parent[i] = i;`

This means every node is its own root.

Example:

`parent[0] = 0`
`parent[1] = 1`
`parent[2] = 2`

After union 0 and 1:

`parent[1] = 0`

Now 0 is representative of $\{0,1\}$.

6. Find Operation

find(x) follows parent pointers until root.

Root is node whose parent is itself:

```
parent[root] == root
```

Naive:

```
int find(int x) {  
    while (parent[x] != x) {  
        x = parent[x];  
    }  
  
    return x;  
}
```

7. Path Compression

Path compression makes future find calls faster.

If:

```
0 <- 1 <- 2 <- 3
```

Then find(3) discovers root 0.

Path compression updates:

```
parent[3] = 0  
parent[2] = 0  
parent[1] = 0
```

Recursive implementation:

```
int find(int x) {  
    if (parent[x] != x) {  
        parent[x] = find(parent[x]);  
    }  
  
    return parent[x];  
}
```

This flattens the tree.

8. Union by Rank / Size

If we always attach randomly, trees can become tall.

Union by size attaches smaller tree under larger tree.

```
if (size[rootA] < size[rootB]) {  
    swap(rootA, rootB);  
}
```

```
}
```

```
parent[rootB] = rootA;  
size[rootA] += size[rootB];
```

This keeps trees shallow.

9. Optimized Complexity

With:

- path compression
- union by size/rank

Operations are almost constant time.

Technically:

$O(\alpha(n))$

where alpha is inverse Ackermann function.

For practical input sizes, this is nearly $O(1)$.

10. When to Use Union Find

Use Union Find when:

10.1 Need Connectivity in Undirected Graph

Examples:

- connected components
- path exists
- provinces

10.2 Edges Are Added Dynamically

Examples:

- Number of Islands II
- Redundant Connection
- network connection problems

10.3 Need Detect Cycle in Undirected Graph

If an edge connects two nodes already in same set, it creates a cycle.

Example:

- Redundant Connection
- Graph Valid Tree

10.4 Need Merge Groups

Examples:

- Accounts Merge
- Similar String Groups
- Smallest String With Swaps

10.5 Kruskal MST

Union Find checks whether adding an edge would create a cycle.

11. When NOT to Use Union Find

11.1 Directed Graph Cycle Detection

Use DFS 3-state or topological sort.

Union Find is not suitable for general directed cycle detection.

11.2 Shortest Path

Use BFS/Dijkstra.

Union Find only tells connected or not, not distance.

11.3 Need Actual Path

Use DFS/BFS.

Union Find says connected but cannot reconstruct path.

11.4 Need Dynamic Edge Deletion

Basic Union Find handles additions well, not deletions.

11.5 Need Ordered Traversal

Use graph traversal or ordered structures.

12. Union Find Class Template

```
class DSU {
private:
    vector<int> parent;
    vector<int> size;

public:
    DSU(int n) {
        parent.resize(n);
        size.resize(n, 1);

        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }

    bool unite(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
```

```

    if (rootA == rootB) {
        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];

    return true;
}

bool connected(int a, int b) {
    return find(a) == find(b);
}
};

```

unite returns:

true if merge happened
false if already connected

This is useful for cycle detection.

13. Union Find With Component Count

```

class DSU {
private:
    vector<int> parent;
    vector<int> size;

public:
    int components;

    DSU(int n) {
        components = n;
        parent.resize(n);
        size.resize(n, 1);

        for (int i = 0; i < n; i++) {

```

```

        parent[i] = i;
    }
}

int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }

    return parent[x];
}

bool unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];
    components--;

    return true;
}
};

```

Each successful union reduces component count by 1.

14. Mental Models

14.1 Friend Groups

If A is friends with B, merge their friend groups.

To check if two people are in same friend group, compare group leaders.

14.2 Island Merging

Each land cell starts as an island.

When adjacent land cells appear, merge islands.

14.3 Cycle Detection

If edge connects two nodes already in same group, then that edge closes a loop.

14.4 Representative Leader

Each component has one leader.

find(x) returns that leader.

15. Pattern Recognition Checklist

Ask:

19. Is this undirected connectivity?
20. Are elements being grouped or merged?
21. Are edges added dynamically?
22. Do I need count components?
23. Do I need detect redundant edge?
24. Do I only need know connected/not connected?
25. Is path length irrelevant?
26. Is this Kruskal MST?
27. Are there equivalence relationships?
28. Would DFS/BFS be repeated too many times?

If yes, Union Find may apply.

16. Core Templates

Template 1 — Connected Components From Edges

```
DSU dsu(n);

for (auto& edge : edges) {
    dsu.unite(edge[0], edge[1]);
}

return dsu.components;
```

Template 2 — Path Exists

```
DSU dsu(n);

for (auto& edge : edges) {
    dsu.unite(edge[0], edge[1]);
}

return dsu.connected(source, destination);
```

Template 3 — Redundant Edge

```
for (auto& edge : edges) {
    if (!dsu.unite(edge[0], edge[1])) {
```

```

        return edge;
    }
}

```

Template 4 — Group Elements by Root

```

unordered_map<int, vector<int>> groups;

for (int i = 0; i < n; i++) {
    int root = dsu.find(i);
    groups[root].push_back(i);
}

```

Template 5 — String to ID Mapping

```

unordered_map<string, int> id;

int getId(string s) {
    if (!id.count(s)) {
        id[s] = id.size();
    }

    return id[s];
}

```

17. Major Variations

Component count	successful union decreases count	Connected Components
Cycle detection	union fails if same root	Redundant Connection
Connectivity query	compare roots	Valid Path
Matrix connectivity	union connected cells/nodes	Provinces
String grouping	map strings to IDs	Accounts Merge
Equivalence classes	union equal variables	Equations Possible
Grid dynamic union	add land and union neighbors	Number of Islands II

Variation	Core Idea	Example
Kruskal MST	union edges by weight	Min Cost Connect Points
Row/column grouping	union row and column IDs	Most Stones Removed

18. Full Problem Walkthrough 1 — Number of Connected Components

18.1 Problem Summary

Given n nodes and undirected edges.

Return number of connected components.

18.2 How to Identify Union Find

Need group nodes connected by edges.

Each edge unions two nodes.

Final number of groups is components.

18.3 C++ Code

```
class DSU {
public:
    vector<int> parent;
    vector<int> size;
    int components;

    DSU(int n) {
        parent.resize(n);
        size.resize(n, 1);
        components = n;

        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }
    }
}
```

```

        return parent[x];
    }

    bool unite(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);

        if (rootA == rootB) {
            return false;
        }

        if (size[rootA] < size[rootB]) {
            swap(rootA, rootB);
        }

        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        components--;

        return true;
    }
};

class Solution {
public:
    int countComponents(int n, vector<vector<int>>& edges) {
        DSU dsu(n);

        for (auto& edge : edges) {
            dsu.unite(edge[0], edge[1]);
        }

        return dsu.components;
    }
};

```

18.4 Complexity

Time:

$O((n + m) * \alpha(n))$

Usually treated as near:

$O(n + m)$

Space:

$O(n)$

19. Full Problem Walkthrough 2 — LeetCode 547: Number of Provinces

19.1 Problem Summary

Given city connection matrix.

Return number of connected components.

19.2 How to Identify Union Find

Cities are nodes.

If `isConnected[i][j] == 1`, union `i` and `j`.

19.3 C++ Code

```
class DSU {
public:
    vector<int> parent;
    vector<int> size;
    int components;

    DSU(int n) {
        parent.resize(n);
        size.resize(n, 1);
        components = n;

        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }
}
```

```

void unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];
    components--;
}

};

class Solution {
public:
    int findCircleNum(vector<vector<int>>& isConnected) {
        int n = isConnected.size();

        DSU dsu(n);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (isConnected[i][j] == 1) {
                    dsu.unite(i, j);
                }
            }
        }

        return dsu.components;
    }
};

```

19.4 Complexity

Time:

$O(n^2 * \alpha(n))$

Space:

$O(n)$

20. Full Problem Walkthrough 3 — LeetCode 1971: Find if Path Exists in Graph

20.1 Problem Summary

Given undirected graph, source, destination.

Return true if source and destination are connected.

20.2 How to Identify Union Find

Need connectivity only.

Union all edges.

Check if source and destination have same root.

20.3 C++ Code

```
class Solution {
public:
    class DSU {
    public:
        vector<int> parent;
        vector<int> size;

        DSU(int n) {
            parent.resize(n);
            size.resize(n, 1);

            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }

            return parent[x];
        }
    };
}
```

```

void unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];
}

bool connected(int a, int b) {
    return find(a) == find(b);
}

};

bool validPath(int n,
               vector<vector<int>>& edges,
               int source,
               int destination) {
    DSU dsu(n);

    for (auto& edge : edges) {
        dsu.unite(edge[0], edge[1]);
    }

    return dsu.connected(source, destination);
}

};

```

20.4 Complexity

Time:

$O((n + m) * \alpha(n))$

Space:

$O(n)$

21. Full Problem Walkthrough 4 — LeetCode 684: Redundant Connection

21.1 Problem Summary

Given edges of an undirected graph that started as a tree and then had one extra edge added.

Return the edge that can be removed to make it a tree again.

21.2 How to Identify Union Find

A tree with n nodes has no cycle.

Process edges one by one.

If edge connects two nodes already connected, it creates cycle.

That edge is redundant.

21.3 C++ Code

```
class Solution {
public:
    class DSU {
    public:
        vector<int> parent;
        vector<int> size;

        DSU(int n) {
            parent.resize(n + 1);
            size.resize(n + 1, 1);

            for (int i = 1; i <= n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }

            return parent[x];
        }

        bool unite(int a, int b) {
```

```

    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];

    return true;
}
};

vector<int> findRedundantConnection(vector<vector<int>>& edges) {
    int n = edges.size();

    DSU dsu(n);

    for (auto& edge : edges) {
        if (!dsu.unite(edge[0], edge[1])) {
            return edge;
        }
    }

    return {};
}
};

```

21.4 Complexity

Time:

$O(n * \alpha(n))$

Space:

$O(n)$

21.5 Common Mistake

This works for undirected cycle detection.

For directed redundant connection, problem is harder and needs extra logic.

22. Full Problem Walkthrough 5 — Accounts Merge

22.1 Problem Summary

Accounts with common email belong to same person.
Merge accounts and output sorted emails.

22.2 How to Identify Union Find

Emails that appear in same account should belong to same connected component.
Union emails within each account.
Then group emails by root.

22.3 C++ Code

```
class Solution {
public:
    class DSU {
    public:
        vector<int> parent;
        vector<int> size;

        DSU(int n) {
            parent.resize(n);
            size.resize(n, 1);

            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }

            return parent[x];
        }

        void unite(int a, int b) {
            int rootA = find(a);
            int rootB = find(b);
```

```

        if (rootA == rootB) {
            return;
        }

        if (size[rootA] < size[rootB]) {
            swap(rootA, rootB);
        }

        parent[rootB] = rootA;
        size[rootA] += size[rootB];
    }
};

vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
    unordered_map<string, int> emailId;
    unordered_map<string, string> emailName;

    int id = 0;

    for (auto& account : accounts) {
        string name = account[0];

        for (int i = 1; i < account.size(); i++) {
            string email = account[i];

            if (!emailId.count(email)) {
                emailId[email] = id++;
            }

            emailName[email] = name;
        }
    }

    DSU dsu(id);

    for (auto& account : accounts) {
        int firstId = emailId[account[1]];

        for (int i = 2; i < account.size(); i++) {
            dsu.unite(firstId, emailId[account[i]]);
        }
    }
}

```

```

    }

    unordered_map<int, vector<string>> groups;

    for (auto& [email, idx] : emailId) {
        int root = dsu.find(idx);
        groups[root].push_back(email);
    }

    vector<vector<string>> result;

    for (auto& [root, emails] : groups) {
        sort(emails.begin(), emails.end());

        vector<string> merged;
        merged.push_back(emailName[emails[0]]);

        for (string& email : emails) {
            merged.push_back(email);
        }

        result.push_back(merged);
    }

    return result;
}
};

```

22.4 Complexity

Let E be total number of emails.

Union/find:

$O(E * \alpha(E))$

Sorting:

$O(E \log E)$

Overall:

$O(E \log E)$

23. Full Problem Walkthrough 6 — LeetCode 990: Satisfiability of Equality Equations

23.1 Problem Summary

Equations like:

```
a==b
a!=b
```

Return true if all equations can be satisfied.
Variables are lowercase letters.

23.2 How to Identify Union Find

Equality creates equivalence groups.

First union all equality equations.

Then check inequality equations.

If $a \neq b$ but a and b are in same group, contradiction.

23.3 C++ Code

```
class Solution {
public:
    class DSU {
    public:
        vector<int> parent;

        DSU(int n) {
            parent.resize(n);

            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }

            return parent[x];
        }
    };
}
```

```

void unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA != rootB) {
        parent[rootB] = rootA;
    }
}

};

bool equationsPossible(vector<string>& equations) {
    DSU dsu(26);

    for (string& eq : equations) {
        if (eq[1] == '=') {
            int a = eq[0] - 'a';
            int b = eq[3] - 'a';

            dsu.unite(a, b);
        }
    }

    for (string& eq : equations) {
        if (eq[1] == '!') {
            int a = eq[0] - 'a';
            int b = eq[3] - 'a';

            if (dsu.find(a) == dsu.find(b)) {
                return false;
            }
        }
    }

    return true;
}

};

```

23.4 Complexity

Only 26 variables.

Time:

$O(\text{number of equations})$

Space:

$O(1)$

24. Full Problem Walkthrough 7 — Graph Valid Tree

24.1 Problem Summary

Given n nodes and undirected edges.

Return true if edges form a valid tree.

A valid tree must be:

- 29. connected
- 30. acyclic

24.2 How to Identify Union Find

A tree with n nodes must have exactly:

$n - 1$

edges.

Then use union find to detect cycle and connectivity.

24.3 C++ Code

```
bool validTree(int n, vector<vector<int>>& edges) {  
    if (edges.size() != n - 1) {  
        return false;  
    }  
  
    DSU dsu(n);  
  
    for (auto& edge : edges) {  
        if (!dsu.unite(edge[0], edge[1])) {  
            return false;  
        }  
    }  
  
    return true;  
}
```

Assumes DSU class with unite returning false if already connected.

24.4 Why Edge Count Check Works

For undirected graph:

If there are $n - 1$ edges and no cycle, graph must be connected.

So union cycle check plus edge count is enough.

25. Full Problem Walkthrough 8 — LeetCode 947: Most Stones Removed with Same Row or Column

25.1 Problem Summary

A stone can be removed if another stone shares same row or column.
Return maximum stones removed.

25.2 Key Insight

In each connected component of stones, you can remove all but one stone.
Answer:

stones count - number of connected components

Stones are connected if they share row or column.

25.3 Union Find Approach

Treat each stone as node.

Union stones that share row or column.

Naive pair checking is $O(n^2)$, acceptable for many constraints.

25.4 C++ Code, Pairwise

```
class Solution {
public:
    int removeStones(vector<vector<int>>& stones) {
        int n = stones.size();

        DSU dsu(n);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (stones[i][0] == stones[j][0] ||
                    stones[i][1] == stones[j][1]) {
                    dsu.unite(i, j);
                }
            }
        }

        unordered_set<int> roots;
```

```
    for (int i = 0; i < n; i++) {  
        roots.insert(dsu.find(i));  
    }  
  
    return n - roots.size();  
}  
};
```

25.5 Optimized Row/Column Mapping Idea

Union row ID with column ID.

Need offset columns to avoid collision.

```
row = stones[i][0]  
col = stones[i][1] + 10001
```

Then number of connected components among used row/col nodes determines answer.

26. Full Problem Walkthrough 9 — LeetCode 1319: Number of Operations to Make Network Connected

26.1 Problem Summary

There are n computers and cables.

You can remove extra cable and reconnect elsewhere.

Return minimum operations to connect all computers.

If impossible, return -1.

26.2 Key Insight

To connect n nodes, need at least:

$n - 1$

cables.

If cables fewer, impossible.

Otherwise, answer is:

components - 1

26.3 C++ Code

```

class Solution {
public:
    int makeConnected(int n, vector<vector<int>>& connections) {
        if (connections.size() < n - 1) {
            return -1;
        }

        DSU dsu(n);

        for (auto& edge : connections) {
            dsu.unite(edge[0], edge[1]);
        }

        return dsu.components - 1;
    }
};

```

26.4 Complexity

Time:

$O((n + m) * \alpha(n))$

Space:

$O(n)$

27. Full Problem Walkthrough 10 — Min Cost to Connect All Points, Kruskal Preview

27.1 Problem Summary

Given points in 2D plane.

Cost to connect two points is Manhattan distance.

Return minimum cost to connect all points.

27.2 How Union Find Applies

This is Minimum Spanning Tree.

Kruskal's algorithm:

31. Build all edges with weights.
32. Sort by weight.
33. Add edge if it connects different components.
34. Stop after $n - 1$ edges.

Union Find detects whether edge creates cycle.

27.3 C++ Code

```
class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
        int n = points.size();

        vector<vector<int>> edges;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int cost = abs(points[i][0] - points[j][0]) +
                    abs(points[i][1] - points[j][1]);

                edges.push_back({cost, i, j});
            }
        }

        sort(edges.begin(), edges.end());

        DSU dsu(n);

        int total = 0;
        int used = 0;

        for (auto& edge : edges) {
            int cost = edge[0];
            int u = edge[1];
            int v = edge[2];

            if (dsu.unite(u, v)) {
                total += cost;
                used++;

                if (used == n - 1) {
                    break;
                }
            }
        }

        return total;
    }
};
```

```
}  
};
```

27.4 Complexity

Edges:

$O(n^2)$

Sorting:

$O(n^2 \log n)$

Space:

$O(n^2)$

28. Full Problem Walkthrough 11 — LeetCode 959: Regions Cut By Slashes

28.1 Problem Summary

Grid contains:

```
'/'  
'\  
' '
```

Slashes divide squares into regions.
Return number of regions.

28.2 Union Find Idea

Divide each cell into 4 triangles:

0 = top
1 = right
2 = bottom
3 = left

Union triangles inside cell depending on slash.
Union neighboring cell triangles across borders.
Count final components.

28.3 Internal Cell Rules

If space:

union all 4 triangles

If slash /:

top with left
right with bottom

If backslash \:

top with right
bottom with left

28.4 C++ Sketch

```
int id(int r, int c, int part, int n) {  
    return (r * n + c) * 4 + part;  
}
```

For each cell:

```
if (grid[r][c] == ' ') {  
    unite(0,1), unite(1,2), unite(2,3)  
} else if (grid[r][c] == '/') {  
    unite(0,3), unite(1,2)  
} else {  
    unite(0,1), unite(2,3)  
}
```

Neighbor unions:

bottom of current with top of below
right of current with left of right

29. Full Problem Walkthrough 12 — Similar String Groups

29.1 Problem Summary

Two strings are similar if they are equal or can become equal by swapping two characters.

Return number of similarity groups.

29.2 How to Identify Union Find

Similarity is an equivalence relation.

Union strings that are similar.

Count groups.

29.3 Similarity Check

Two strings are similar if they differ in either:

0 positions

or:

2 positions

29.4 C++ Code

```
class Solution {
public:
    bool similar(string& a, string& b) {
        int diff = 0;

        for (int i = 0; i < a.size(); i++) {
            if (a[i] != b[i]) {
                diff++;
            }
        }

        return diff == 0 || diff == 2;
    }

    int numSimilarGroups(vector<string>& strs) {
        int n = strs.size();

        DSU dsu(n);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (similar(strs[i], strs[j])) {
                    dsu.unite(i, j);
                }
            }
        }

        return dsu.components;
    }
};
```

29.5 Complexity

Let:

n = number of strings
L = string length

Time:

$O(n^2 * L)$

Space:

$O(n)$

30. Full Problem Walkthrough 13 — Smallest String With Swaps

30.1 Problem Summary

Given string and pairs of indices that can be swapped any number of times. Return lexicographically smallest string possible.

30.2 Key Insight

Indices connected by swap pairs form components.

Characters inside each component can be rearranged arbitrarily.

For each component:

- collect indices
 - collect characters
 - sort both
 - assign smallest characters to smallest indices
-

30.3 C++ Code

```
class Solution {
public:
    string smallestStringWithSwaps(string s, vector<vector<int>>& pairs) {
        int n = s.size();

        DSU dsu(n);

        for (auto& pair : pairs) {
            dsu.unite(pair[0], pair[1]);
        }

        unordered_map<int, vector<int>> groups;

        for (int i = 0; i < n; i++) {
            groups[dsu.find(i)].push_back(i);
        }

        string result = s;

        for (auto& [root, indices] : groups) {
            vector<char> chars;
```

```

        for (int idx : indices) {
            chars.push_back(s[idx]);
        }

        sort(indices.begin(), indices.end());
        sort(chars.begin(), chars.end());

        for (int i = 0; i < indices.size(); i++) {
            result[indices[i]] = chars[i];
        }
    }

    return result;
}
};

```

30.4 Complexity

Time:

$O(n \log n)$

due to sorting groups.

Space:

$O(n)$

31. Full Problem Walkthrough 14 — Number of Islands II

31.1 Problem Summary

Initially grid is all water.

Land cells are added one by one.

After each addition, return number of islands.

31.2 How to Identify Union Find

This is dynamic connectivity.

Each new land cell creates a new component.

Union with adjacent land cells.

Each successful union reduces island count.

31.3 C++ Code

```

vector<int> numIslands2(int m,
                        int n,
                        vector<vector<int>>& positions) {

```

```

DSU dsu(m * n);

vector<vector<int>> grid(m, vector<int>(n, 0));
vector<int> result;

int islands = 0;

vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};

auto getId = [&](int r, int c) {
    return r * n + c;
};

for (auto& pos : positions) {
    int r = pos[0];
    int c = pos[1];

    if (grid[r][c] == 1) {
        result.push_back(islands);
        continue;
    }

    grid[r][c] = 1;
    islands++;

    int id1 = getId(r, c);

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nc < 0 || nr >= m || nc >= n) {
            continue;
        }

        if (grid[nr][nc] == 0) {
            continue;
        }
    }
}

```

```

    }

    int id2 = getId(nr, nc);

    if (dsu.unite(id1, id2)) {
        islands--;
    }
}

result.push_back(islands);
}

return result;
}

```

31.4 Complexity

For k additions:

Time:

$O(k * \alpha(mn))$

Space:

$O(mn)$

32. Common Edge Cases for Union Find

32.1 Already Connected

unite should return false and not reduce components.

32.2 Self Union

unite(x, x) should return false.

32.3 1-Indexed Nodes

Allocate $n + 1$.

32.4 Empty Edges

Components remain n.

32.5 Duplicate Edges

Second duplicate union returns false.

May indicate redundant edge depending on problem.

32.6 Dynamic Land Duplicate Additions

Number of Islands II must handle adding same land cell again.

32.7 String Nodes

Map strings to integer IDs.

33. Common Mistakes and Wrong Code

Mistake 1 — No Path Compression

Can cause slow performance on large tests.

Mistake 2 — Not Calling Find Before Union

Wrong:

```
parent[b] = a;
```

Correct:

```
rootA = find(a);  
rootB = find(b);  
parent[rootB] = rootA;
```

Mistake 3 — Decreasing Component Count Even When Already Connected

Only decrement when roots differ.

Mistake 4 — Using Union Find for Directed Cycle Detection

Use DFS/topological sort instead.

Mistake 5 — Forgetting 1-Indexed Allocation

For nodes 1..n, use:

```
DSU dsu(n + 1);
```

or internal arrays size $n + 1$.

Mistake 6 — Not Grouping by Final Root

When building groups, call:

```
dsu.find(i)
```

not just:

```
parent[i]
```

because parent may not be compressed.

34. Interview Explanation Templates

34.1 Union Find General

I use Union Find to maintain connected components. Each node starts in its own component. For every connection, I union the two components. To check whether two nodes are connected, I compare their roots.

34.2 Cycle Detection

In an undirected graph, if an edge connects two nodes already in the same component, adding that edge creates a cycle.

34.3 Component Count

I initialize the component count to n . Each successful union merges two components, so I decrement the count.

34.4 Accounts Merge

I treat emails as nodes. Emails in the same account are unioned. Then I group emails by their root parent and sort each group.

34.5 Dynamic Islands

Each new land cell starts as a new island. Then I union it with adjacent land cells. Each successful union reduces the island count by one.

35. Complexity Patterns

Union Find operation	$O(\alpha(n))$	$O(n)$
Connected components	$O((n + m) \alpha(n))$	$O(n)$
Provinces	$O(n^2 \alpha(n))$	$O(n)$
Valid path	$O((n + m) \alpha(n))$	$O(n)$
Redundant Connection	$O(n \alpha(n))$	$O(n)$
Accounts Merge	$O(E \log E)$	$O(E)$
Equations Possible	$O(E)$	$O(1)$
Graph Valid Tree	$O(m \alpha(n))$	$O(n)$
Most Stones Removed	$O(n^2 \alpha(n))$ pairwise	$O(n)$
Network Connected	$O(m \alpha(n))$	$O(n)$
Kruskal MST	$O(E \log E)$	$O(V)$
Number of Islands II	$O(k \alpha(mn))$	$O(mn)$
Similar String Groups	$O(n^2 L)$	$O(n)$
Smallest String With Swaps	$O(n \log n)$	$O(n)$

36. Time-Boxed Practice Guidance

Intro Union Find

Target:

15–25 minutes

Examples:

- Number of Connected Components
- Find if Path Exists
- Number of Provinces

Medium Union Find

Target:

25–45 minutes

Examples:

- Redundant Connection
- Accounts Merge
- Satisfiability of Equality Equations
- Graph Valid Tree
- Network Connected

Harder Union Find

Target:

45–75 minutes

Examples:

- Most Stones Removed
- Smallest String With Swaps
- Number of Islands II
- Regions Cut By Slashes
- Min Cost to Connect All Points

37. Expanded Practice Set

37.1 Must-Do Medium

35. Number of Connected Components in an Undirected Graph
36. Number of Provinces
37. Find if Path Exists in Graph
38. Redundant Connection
39. Accounts Merge
40. Satisfiability of Equality Equations
41. Graph Valid Tree
42. Number of Operations to Make Network Connected
43. Most Stones Removed with Same Row or Column
44. Smallest String With Swaps

37.2 Strong Interview Practice

45. Min Cost to Connect All Points
46. Similar String Groups
47. Regions Cut By Slashes

- 48. Number of Islands II
- 49. Couples Holding Hands
- 50. Swim in Rising Water
- 51. Last Day Where You Can Still Cross
- 52. Remove Max Number of Edges to Keep Graph Fully Traversable

37.3 Hard / Stretch

- 53. Remove Max Number of Edges to Keep Graph Fully Traversable
 - 54. Last Day Where You Can Still Cross
 - 55. Number of Islands II
 - 56. Regions Cut By Slashes
 - 57. Bricks Falling When Hit
 - 58. Checking Existence of Edge Length Limited Paths
 - 59. Making A Large Island
 - 60. Rank Transform of a Matrix
-

38. Suggested Study Order

Step 1 — Basic DSU Operations

Start with:

- 61. Implement DSU
- 62. Find if Path Exists
- 63. Number of Connected Components

Goal:

Master find, unite, and connected.

Step 2 — Cycle and Tree Checks

Solve:

- 64. Redundant Connection
- 65. Graph Valid Tree

Goal:

Use failed union to detect cycles.

Step 3 — Grouping Problems

Solve:

- 66. Accounts Merge
- 67. Similar String Groups
- 68. Smallest String With Swaps

Goal:

Union related objects, then group by root.

Step 4 — Connectivity With Constraints

Solve:

- 69. Equations Possible
- 70. Network Connected
- 71. Most Stones Removed

Goal:

Recognize equivalence and component-count patterns.

Step 5 — Advanced DSU

Solve:

- 72. Number of Islands II
- 73. Regions Cut By Slashes
- 74. Min Cost Connect Points

Goal:

Use DSU for dynamic grids and MST.

39. Pattern Decision Table

connected components	union edges	Components
path exists	compare roots	Valid Path
undirected cycle	failed union	Redundant Connection
valid tree	edges = $n - 1$ + no failed union	Graph Valid Tree
merge accounts	emails as nodes	Accounts Merge
equality groups	union equal variables	Equations Possible
dynamic islands	add land + union neighbors	Number of Islands II
can remove stones	components among stones	Most Stones
network cables	components - 1	Network Connected
MST	sort edges + union	Kruskal
swaps allowed	components of indices	Smallest String Swaps
string similarity	union similar strings	Similar Groups

40. Final Cheat Sheet

DSU Class

```
class DSU {  
public:  
    vector<int> parent;  
    vector<int> size;  
    int components;
```

```

DSU(int n) {
    parent.resize(n);
    size.resize(n, 1);
    components = n;

    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }
}

int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }

    return parent[x];
}

bool unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];
    components--;

    return true;
}

bool connected(int a, int b) {
    return find(a) == find(b);
}
};

```

Connected Components

```
DSU dsu(n);

for (auto& edge : edges) {
    dsu.unite(edge[0], edge[1]);
}

return dsu.components;
```

Redundant Edge

```
if (!dsu.unite(u, v)) {
    return {u, v};
}
```

Group by Root

```
unordered_map<int, vector<int>> groups;

for (int i = 0; i < n; i++) {
    groups[dsu.find(i)].push_back(i);
}
```

String ID Mapping

```
if (!id.count(s)) {
    id[s] = id.size();
}
```

41. What to Master Before Chapter 29

Before moving to Shortest Path, make sure you can:

- Implement DSU from memory
- Explain find
- Explain union
- Explain path compression
- Explain union by size/rank
- Use DSU for connected components
- Use DSU for valid path
- Use DSU for undirected cycle detection
- Know why DSU is not for directed cycle detection
- Count components correctly
- Group elements by root
- Map strings to IDs
- Solve Redundant Connection

- Solve Accounts Merge
 - Solve Equations Possible
 - Solve Number of Islands II
 - Understand Kruskal preview
-

42. Core Takeaway

Union Find manages connected components.

It is built around:

find root
union roots
compare roots

Use it when connectivity is about undirected grouping and edges are being added or processed.

It is excellent for:

- connected components
- redundant edge detection
- grouping equivalent items
- dynamic island counting
- Kruskal MST

It is not for:

- shortest path
- directed cycle detection
- actual path reconstruction
- general graph traversal

The most important implementation rule is:

Always union roots, not raw nodes.

With path compression and union by size, Union Find is extremely fast and simple.